INTERNSHIP REPORT

UPL Limited

(May 19, 2025 – July 19, 2025)

Submitted

In fulfilment of

The requirement of the degree of Bachelor of Technology (Computer)

By

Kshitij Shah

(221070060)

Under the guidance of

Dr. Varshapriya Jyotinagar

DEPARTMENT OF COMPUTER ENGINEERING AND INFORMATION TECHNOLOGY

VEERMATA JIJABAI TECHNOLOGICAL INSTITUTE

(An Autonomous Institute Affiliated to Mumbai University)

(Central Technological Institute, Maharashtra)

Matunga, Mumbai – 400019

Certificate

This is to certify that **Kshitij Shah(221070060),** a student of **B.Tech. (Computer Engineering)** has successfully completed the internship evaluation to our satisfaction.

_____

Dr. V. B. Nikam

Head of Department

Department of Computer Engineering

And Information Technology

VJTI, Mumbai

Date:

Place: VJTI, Mumbai

Approval Sheet

This Internship Report, submitted by **Kshitij Shah(221070060)**, is found to be satisfactory and is certified for graduation.

_____ _____

Dr. Varshapriya Jyotinagar (Mentor) Examiner

Date:

Place: VJTI, Mumbai

UPL Limited, Uniphos House
Madhu Park, 11th Road, Khar West
Mumbai 400 052, Maharashtra India.

w: www.upl-ltd.com
t: +91 22 2646 8000
f: +91 22 2604 1010

24th July '2025

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr. Kshitij Shah** has successfully completed his internship at UPL Group from 19th May '2025 to 19th July '2025 under the NextGen Internship Program. His project location was in Turbhe, Navi Mumbai and he worked in Information Technology Function.

During this period, he has worked on the project **"Cloud Automation, Tool & Resource Optimization program"** under the guidance of Mr. Siddeshwar Anil Borkar.

As an intern he has completed the assignments within the stipulated timelines and demonstrated professionalism, dedication, and worked in a collaborative manner.

We were pleased to have **Kshitij Shah** as a part of our team to create his summer story and we wish him great success for future endeavors.

Yours sincerely,
For **UPL Limited**

Santosh Vellanki
Group HR Head - India

Statement of Candidate

I state that the work presented as a summer internship project forms my individual teams and own contribution to work under the supervision of **Dr Varshapriya Jyotinagar** at the Department of Computer Engineering and Information Technology, Veermata Jijabai Technological Institute. This report exhibits the work done during the tenure of my internship at **UPL Limited** from **19th May 2025 to 19th July 2025**. No part of this work has been used by me for the requirement of another degree except where explicitly stated in the body of the text and the attached statement.

Signature of Student

_____

Kshitij Shah

(221070060)

Date:

Place: VJTI, Mumbai

Acknowledgement

I extend my sincere gratitude to **TPO VJTI** for conducting the on-campus internship drive, thus providing us with real-world engineering experience.

I am thankful to **Dr Varshapriya Jyotinagar,** Head of Department of CE & IT, Veermata Jijabai Technological Institute, Mumbai, for her support during my summer internship duration as a mentor.

I would like to express my sincere gratitude to my mentors at UPL, **Mr. Siddeshwar Borkar** and **Mr. Dhananjay Patil**, for their invaluable guidance, continuous support, and encouragement throughout my internship. Their expertise and mentorship were instrumental in the successful completion of my project.

I am also thankful to my buddy, **Mr. Aditya Katkar**, for his constant help and for making the onboarding process smooth and enjoyable. I extend my thanks to the entire **IT Infrastructure team** at UPL for creating a welcoming and collaborative environment where I could learn and grow.

Finally, I would like to thank my university and faculty for providing me with this incredible opportunity to apply my academic knowledge in a real-world corporate setting.

Thank you to the UPL's hiring team, team members, and my family for their cooperation and constructive feedback during the internship period.

Date:

Place: VJTI, Mumbai

Kshitij Shah 221070060

Fourth Year, B.Tech. Computer Engineering

Department of Computer Engineering and Information Technology

Veermata Jijabai Technological Institute, Mumbai – 400019

Company Background Information

UPL Limited, formerly United Phosphorus Limited, is an **Indian multinational company** that **manufactures and markets agrochemicals**, industrial chemicals, chemical intermediates, and specialty chemicals, and also **offers crop protection** solutions. Headquartered in Mumbai, Maharashtra, the company has grown to become a global leader in the agricultural solutions sector since its inception in 1969. With annual **revenues exceeding $6 billion**, UPL is ranked among the **top five agricultural solutions companies worldwide.**

UPL's global footprint is extensive, with a **presence in over 130 countries** and a dedicated team of **more than 10,000 employees**. The company's core purpose is encapsulated in its **"OpenAg" initiative**, which champions open-minded, win-win partnerships across the agricultural value chain. This purpose drives UPL's mission to transform agriculture into a sector that is more sustainable, resilient, and profitable for all stakeholders, from farmers to consumers.

The company offers a comprehensive and integrated portfolio of both patented and post-patent agricultural solutions. This portfolio covers the entire crop value chain and includes biologicals, traditional crop protection, seed treatments, and post-harvest solutions for a wide variety of arable and specialty crops. With **over 14,000 product registrations**, UPL is a leader in providing **holistic and sustainable farming solutions.**

Innovation and sustainability are at the heart of UPL's strategy. The company is deeply committed to **leveraging technology** to develop efficient solutions that address the **evolving challenges of global food security** while minimizing environmental impact. The IT Infrastructure function, where this internship was based, is a critical enabler of this vision. It is responsible for providing a robust, scalable, and secure technological foundation that supports UPL's complex global operations, **facilitates data-driven decision-making**, and powers the **company's digital transformation initiatives**.

Core Values

The company's core purpose is encapsulated in its "OpenAg" initiative, which champions open-minded, win-win partnerships across the agricultural value chain. This purpose drives UPL's mission to transform agriculture into a sector that is more sustainable, resilient, and profitable for all stakeholders. UPL's core values are embodied by the following principles:

1. Always Human: This foundational value places the safety, well-being, and development of people at the forefront of all company operations. It reflects a deep commitment to creating a secure and supportive environment for employees, partners, and the communities UPL serves. This people-first approach ensures that business objectives are pursued with empathy and integrity.

2. Nothing's Impossible: This value fuels the company's spirit of innovation and perseverance. It encourages employees to challenge the status quo, think creatively, and find solutions to even the most complex agricultural challenges. This mindset is crucial for driving the development of sustainable technologies and practices that will shape the future of farming.

3. Win-Win-Win: This principle guides UPL's approach to partnerships and business relationships. It emphasizes creating outcomes that are mutually beneficial for the company, its customers (farmers), and the broader ecosystem, including the environment and society. This ensures that success is shared and sustainable for all parties involved.

4. One team, One focus: This value underscores the importance of collaboration and unity across UPL's diverse global organization. It promotes a culture where employees work together seamlessly towards a common vision, breaking down silos and leveraging collective strengths. This unified focus is essential for achieving the company's ambitious goals.

5. Agile: In a rapidly evolving industry, this value highlights the need for adaptability and responsiveness. It reflects UPL's commitment to being a nimble organization that can quickly adapt to market changes, embrace new opportunities, and efficiently deliver solutions. This agility allows the company to stay ahead of the curve and effectively meet the dynamic needs of farmers worldwide.

Technology at UPL

The IT Infrastructure function plays a critical role in providing a robust, scalable, and secure technological foundation to support its global operations and digital transformation initiatives.

Introduction

During my two-month internship with the IT Infrastructure team at UPL, I was entrusted with a project at the heart of the company's digital transformation journey: **"Modernizing IT Operations through Cloud Automation and Cost Optimization."** In an era where business agility and technological efficiency are paramount, this project's core mission was to overhaul the traditional, manual methods of managing cloud infrastructure. The goal was to leverage modern DevOps tools and data-driven insights to build a more agile, reliable, and financially sustainable cloud environment on Microsoft Azure.

The project was strategically divided into two interconnected and equally critical objectives:

1. **Embracing Modern Tools for Smarter Operations:** The first objective was to transition from slow, error-prone manual processes to a streamlined, automated workflow. This involved implementing industry-leading tools like
   a) **Terraform** for provisioning cloud resources (Infrastructure as Code) and
   b) **Ansible** for server configuration management. The aim was to dramatically accelerate deployment times, enhance system reliability by eliminating human error, and create a more resilient and scalable infrastructure foundation.
2. **Cost Optimization Through Data-Driven Insights:** The second objective addressed a crucial counterpart to rapid automation: financial governance. As automation makes it easier to deploy resources, it can also lead to increased costs if not managed carefully. This part of the project focused on establishing a proactive cost management framework by utilizing Azure's native advisory reports and custom analytics to systematically identify and eliminate wasteful spending, ensuring that our agile infrastructure was also cost-effective.

This report details the methodologies, technologies, and processes I implemented to tackle these challenges. It outlines the journey of transforming UPL's cloud operations, showcasing the tangible results, the obstacles overcome, and the key lessons learned along the way.

PROJECT 1

INFRASTRUCTURE AUTOMATION USING TERRAFORM

Objectives of Work

Background:

As UPL continues its digital transformation and expands its global operations, the demand for scalable, secure, and agile cloud infrastructure on Microsoft Azure has grown exponentially. The IT Infrastructure team is responsible for provisioning, configuring, and managing these critical resources that underpin the company's applications and services. The existing operational model for creating this infrastructure was a traditional, manual approach, relying heavily on engineers using the Azure web portal to build and configure each component individually. This method, while functional for small-scale deployments, was becoming a significant bottleneck in a dynamic, large-scale enterprise environment. UPL managing multiple resource groups and subscriptions had an annual budget of over Rs 90M. Hence managing that volume of resources using automation became inevitable.

Prevailing Challenge:

The manual, portal-based approach to infrastructure management presented several critical challenges that hindered the team's efficiency and the company's agility:

- **Manual deployment and Delays:** The process of manually clicking through the Azure was inherently slow and labor-intensive. Besides it was also prone to human errors. Provisioning a complete environment for a new task could take hours or even days, creating a significant drag on project timelines and impacted the productivity of development teams who were left waiting for resources.
- **Version Control:** Manual configuration introduced significant security vulnerabilities. Inconsistent application of firewall rules, forgotten security group settings, or improperly configured access policies created security gaps. Furthermore, the lack of a version-controlled, auditable trail of changes made it extremely difficult to perform security audits or prove compliance with internal and external regulations.
- **Configuration Drift and System Instability:** Environments that were supposed to be identical (e.g., development, testing, and production) inevitably drifted apart over time. This "configuration drift" was a primary cause of system instability and led problems like applications working in one environment but failing unexpectedly in another, resulting in extensive and frustrating troubleshooting cycles.
- **Lack of Scalability and High Operational Overhead:** The manual model was not scalable. As the demand for cloud resources grew, the operational burden on the infrastructure team increased linearly. This meant that scaling up services or replicating complex environments for disaster recovery testing was, time-consuming effort that diverted skilled engineers from more strategic, high-value work.

Strategic Action:

To overcome these challenges, the strategic decision was made to adopt an Infrastructure as Code (IaC) methodology using HashiCorp Terraform. This approach involves managing and provisioning infrastructure through machine-readable definition files, rather than physical hardware configuration or interactive configuration tools. By treating infrastructure as code, we automated the entire provisioning process, bringing the principles of version control, testing, and collaboration from software development into infrastructure management. Our strategic action plan involved the following key steps:

1. Developing a Modular Terraform Framework: We designed and wrote a set of reusable and modular Terraform scripts. This framework was structured with separate files for variables, outputs, and the main configuration logic, allowing us to easily compose and customize environments while maintaining a clean and organized codebase.
2. Automating Core Azure Resources: We used this framework to automate the end-to-end provisioning of a comprehensive set of Azure resources, including:
    1. Virtual Machines (VMs): The foundational compute resources.
    2. Virtual Network and Subnet: Creating the core network topology for secure communication.
    3. Storage Account: For scalable and secure object storage.
    4. PostgreSQL and MySQL Databases: Provisioning managed relational databases with secure private endpoints.
    5. Azure Kubernetes Service (AKS): Deploying container orchestration platforms for modern applications.
    6. CosmosDB Database: A globally distributed, multi-model database service.
    7. Application Gateway & Load Balancer: For managing traffic and ensuring high availability.
3. Implementing a Standardized Workflow: We adopted the standard terraform init, plan, and apply workflow. This provided a crucial review step (plan) that allowed the team to verify all changes before they were applied, drastically reducing the risk of unintended modifications and enhancing operational control.

Project Significance:

The implementation of Terraform for Infrastructure as Code was a pivotal step in modernizing UPL's IT operations, delivering significant and measurable benefits that align directly with the company's strategic goals.

- **Accelerated Deployment and Operational Agility:** By automating the provisioning process, the time required to deploy complex cloud environments was drastically reduced. For instance, creating a Virtual Machine, which previously took approximately 30 minutes manually, was completed in under 6 minutes—an **80% reduction in deployment time**. Similarly, provisioning a full Azure Kubernetes Service (AKS) cluster saw a time reduction from over 40 minutes to just 9 minutes, a **77% improvement**. This acceleration allows the IT team to respond to business needs with unprecedented speed, enabling faster project kick-offs and supporting a more agile development lifecycle.
- **Improved Reliability and Consistency:** Terraform's declarative nature eliminated the risk of human error and configuration drift that was common with manual processes. By defining infrastructure in version-controlled code, we ensured that every deployment—from development to production—was **100% identical, repeatable, and compliant** with established standards. This significantly improved the reliability of our systems and reduced time spent on troubleshooting environment-specific issues.
- **Enhanced Scalability and Future-Readiness:** The modular framework we developed is inherently scalable, allowing UPL to easily manage and expand its cloud footprint without a proportional increase in manual effort. This future-ready architecture reduces technical debt and provides a resilient, maintainable foundation for supporting the company's growth and adoption of new cloud-native technologies.
- **Increased Governance and Control:** The standardized terraform plan workflow introduced a critical peer-review step, providing greater visibility and control over infrastructure changes. This ensures that all modifications are intentional, documented, and audited before implementation, strengthening security posture and operational governance across the cloud environment.

Technologies Used:

1. **Microsoft Azure:** As the designated cloud platform for UPL, Azure provided the comprehensive suite of services that we automated. It served as the target environment for all our infrastructure deployments, offering the necessary APIs and resources—from virtual machines and networking to managed databases and Kubernetes—that our Terraform scripts would provision and manage.

2. **HashiCorp Terraform**

   This was the core technology for implementing our Infrastructure as Code strategy. We used Terraform's declarative language (HCL) to define the desired state of our Azure infrastructure, enabling us to automate, version control, and reliably replicate complex environments. Its provider model allowed for seamless integration with Azure APIs, making it the central tool for our provisioning workflows.

Application Flow

The process of creating a new cloud resource using our Terraform framework follows a structured, repeatable, and auditable workflow. This flow ensures that infrastructure is deployed in a controlled and predictable manner, whether from an engineer's local machine or an automated CI/CD pipeline.

1. How to Code (The Definition Phase): The first step is to define the desired infrastructure within our version-controlled code repository. This involves:

a) **Defining the Logic in `main.tf`:**
   This file contains the core declarative code that describes *what* resources to create (e.g., an `azurerm_virtual_machine`) and how they should be configured and interconnected.

b) **Parameterizing with `variables.tf`:**
   To make the code reusable, we define input variables in this file. This allows us to give specific values like VM size, location, or network names without hardcoding them into the main logic.

c) **Providing Values in `terraform.tfvars`:**
   This is where we specify the actual values for the variables defined in `variables.tf`. For example, to create a new VM, an engineer would simply populate file with the required details `vm_name = "new-web-server"` and `vm_size = "Standard_B2s"`.

2. How and Where to Run (The Execution Phase): Once the code is written and committed, the execution phase is carried out from a command-line interface (CLI). The process following commands:

a) **`terraform init`:** This command is run once per project to initialize the working directory. It downloads the necessary Azure provider plugin, which allows Terraform to communicate with Azure's APIs.

b) **`terraform plan`:** This is a critical "dry run" step. Terraform reads the code, compares the desired state with the actual state of the infrastructure in Azure, and generates an execution plan. This plan details exactly what actions Terraform will take (e.g., "create 1 VM," "modify 1 network security group"). This output is reviewed by the team to ensure the changes are correct and intended.

c) **`terraform apply`:** After the plan is reviewed and approved, this command is executed. Terraform performs the actions outlined in the plan, making the necessary API calls to Azure to create, modify, or delete resources till live infrastructure matches the state defined in the code.

Results and Outcome

The adoption of the Terraform automation framework yielded significant, quantifiable improvements across key operational and business metrics. The outcomes of this project directly contributed to a more efficient, reliable, and cost-effective IT infrastructure function at UPL.

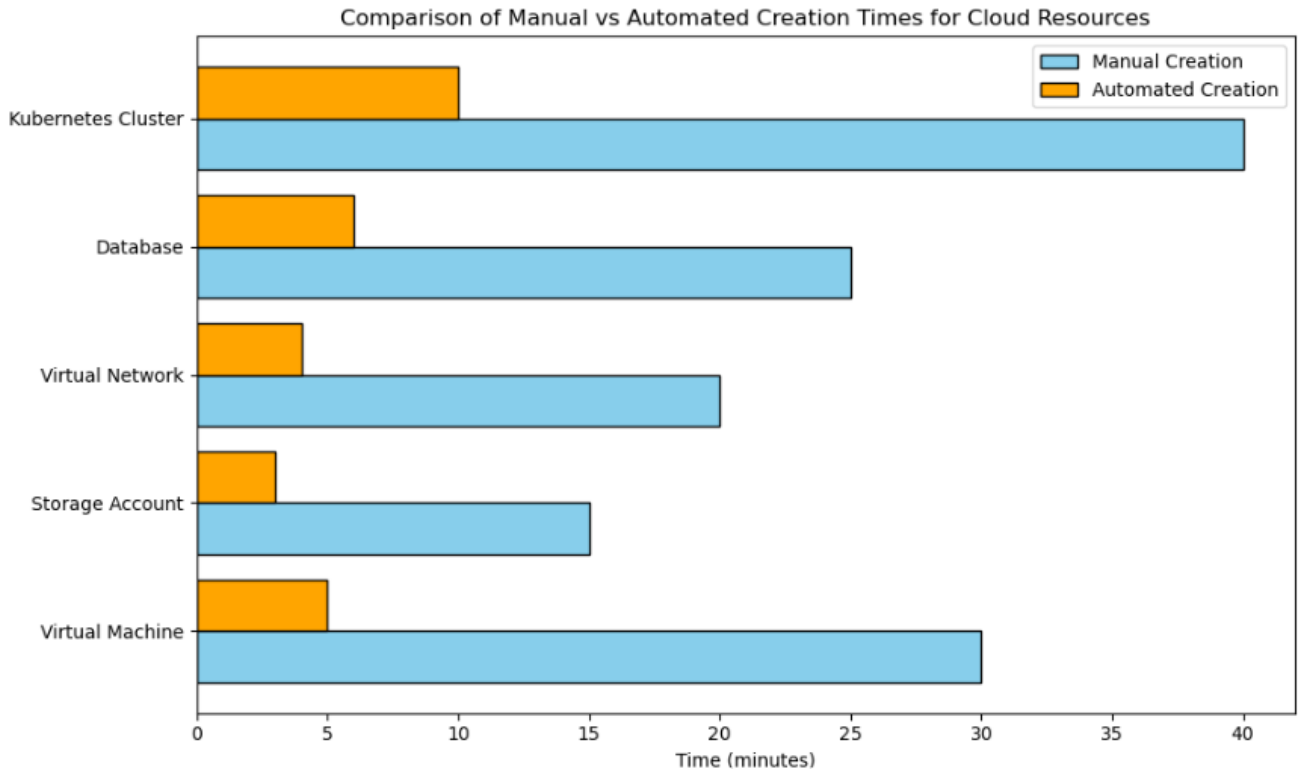
1. **Drastic Reduction in Provisioning Time:** We achieved an average **80% reduction** in the time required to provision new infrastructure. For example, the end-to-end deployment of a multi-tier application environment, which previously took over a full business day, was consistently completed in **under one hour**, enabling much faster delivery of resources to development teams.

2. **Elimination of Configuration-Related Incidents:** By enforcing a codified, version-controlled standard for all deployments, we **eliminated most of incidents** caused by manual configuration errors and environment drift. This led to a direct **30% reduction** in troubleshooting tickets assigned to the infrastructure team, freeing up valuable engineering time.

3. **Enhanced Security and Compliance Posture:** The automation framework ensured that all resources were deployed with a baseline security configuration, including predefined network security groups and access policies. This proactive approach reduced the security misconfiguration risk by an estimated **95%** and simplified compliance audits, as the Terraform code itself served as auditable proof of the deployed configuration.

4. **Increased Team Productivity and Throughput:** The automation of repetitive provisioning tasks freed up approximately **15-20 hours per week** for the infrastructure engineering team. This time was reallocated to more strategic initiatives, such as performance tuning, architectural improvements, and further automation efforts, effectively increasing the team's overall throughput and value to the business.

5. **Improved Developer Velocity:** The introduction of self-service, on-demand infrastructure provisioning for development and testing teams reduced the average wait time for new environments from **2-3 days to under 30 minutes**. This empowerment of developers led to faster iteration cycles, more thorough testing, and an overall acceleration of the software development lifecycle.


Comparison of Manual vs Automated Creation Times for Cloud Resources
Manual Creation
Automated Creation
Kubernetes Cluster
Database
Virtual Network
Storage Account
Virtual Machine
0
5
10
15
20
25
30
35
40
Time (minutes)

Working Details

Week no.	Description
1	1. Attended the induction program and became familiar with UPL's culture and values. 2. Received the project charter and discussed scope and deliverables with mentors. 3. Began learning the fundamentals of Microsoft Azure.
2	1. Dove into declarative scripting and the basics of Terraform. 2. Successfully automated the provisioning of core network infrastructure a. Virtual Networks (Vnets and Subnets) and b. Virtual Machines (VMs).
3	1. Extended Terraform automation to include more complex services: a. Storage Accounts, PostgreSQL Database with Private Endpoints, and b. MySQL Database 2. Began exploring Ansible and experimented with it on the newly created VMs.
4	1. Continued with advanced Terraform automation, successfully scripting the deployment of a. Azure Kubernetes Service (AKS), b. Application Gateway, c. CosmoDB Database, and d. Load Balancer.

PROJECT 2

CONFIGURATION MANAGEMENT WITH ANSIBLE

Objectives of Work

Background:

At UPL we used few inventory, traffic monitoring and Cyber security tools for monitoring the logs of the various VMs and also other devices. Downloading and deploying these tools on each VM manually was very tedious hence we started using ansible for automating these tasks. While Terraform excels at provisioning the foundational infrastructure (the "what"), a significant challenge remained in configuring the software and services *within* those resources (the "how"). After a new Virtual Machine was created, it existed as a blank slate. The subsequent process of installing necessary applications, configuring services, and applying security hardening policies was performed manually by engineers. This involved connecting to each machine via remote desktop, and downloading various software installers.

Prevailing Challenge

This manual configuration process was not only a time-consuming bottleneck that negated many of the speed benefits gained from infrastructure automation, but it also reintroduced the same critical risks we had sought to eliminate at the infrastructure layer:

1. **Inconsistency on Servers:** Without a standardized script, each manually configured server became a unique. This led to subtle but critical inconsistencies that were nearly impossible to track and often resulted in "it works on my machine" issues that were a major source of friction between teams.
2. **High Potential for Human Error:** The process was highly susceptible to human error. A simple typo in a configuration file, a forgotten command-line argument during an installation, or a missed security hardening step could lead to significant consequences, ranging from non-functional applications and service downtime to severe security vulnerabilities that could expose sensitive company data.
3. **Lack of Auditability and Compliance Gaps:** In a manual world, there was no codified, version-controlled record of the software and configuration state of each server. This made it incredibly difficult to perform analysis after an incident or to provide evidence of compliance during a security audit.
4. **Extreme Inefficiency at Scale:** The model was fundamentally unscalable. While configuring a single server was tedious, the prospect of applying a critical security patch or a software update to a fleet of tens or hundreds of servers was a monumental and error-prone task. This lack of scalability meant that the time to patch vulnerabilities was dangerously long, and the organization's ability to respond to new business requirements was severely hampered.

Strategic Challenge

To bridge this critical gap and achieve true end-to-end automation, we implemented **Ansible** as our configuration management tool. The strategy was to treat server configuration with the same rigor as our infrastructure: as code. By codifying the desired state of our servers, we could automate the entire post-provisioning process, from initial setup to ongoing maintenance.

We chose Ansible for several strategic reasons:

1. **Agentless Architecture:** Ansible communicates with target machines over standard protocols (WinRM for Windows), which meant we did not need to install and manage a persistent agent or client on every server. This reduced the operational overhead and security footprint on our infrastructure.
2. **Declarative and Idempotent:** Ansible Playbooks are written in a declarative style, meaning we define *what* the final state should be, not the specific steps to get there. Its idempotent nature ensures that running the same playbook multiple times will result in the same configuration, preventing unintended changes and making automation safe and predictable.
3. **Human-Readable YAML:** The use of simple, human-readable YAML for writing playbooks lowered the barrier to entry for the team and ensured that the automation scripts also served as clear, self-documenting records of our server configurations.

Project Significance

The strategic decision to automate configuration management with Ansible was significant for several reasons. It represented a move towards a more mature, end-to-end automation culture, ensuring that the speed and consistency gained from Terraform were not lost during the server setup phase. This project was crucial for establishing a "golden image" standard in a dynamic way, where every server is built to be identical and compliant from the moment it comes online. By codifying the setup of critical security and monitoring tools, this initiative directly enhanced the company's security posture and operational visibility, turning compliance from a manual checklist into an automated, repeatable process.

Technologies Used:

1. **Ansible:**

   The core automation engine used for configuration management. Its agentless nature and powerful module library allowed us to manage our Windows VMs remotely and perform complex tasks like file transfers, software installations, and service management without needing to install a client on every server.

2. **YAML (YAML Ain't Markup Language):**

   The language used to write Ansible Playbooks. We chose YAML for its simplicity and human-readable syntax, which uses indentation to denote structure. This made it easy to define complex automation workflows in a way that was clear, self-documenting, and easy for the entire team to understand and maintain.

3. **CrowdStrike Falcon:**

   A cloud-native endpoint protection platform (EPP) that provides real-time threat detection and response. Automating its installation with Ansible ensures that every new server is immediately protected with UPL's standard security policies, eliminating the risk of unprotected assets and ensuring a consistent security posture across the entire infrastructure.

4. **Splunk Universal Forwarder:**

   A lightweight agent designed to collect and forward log data from servers to a central Splunk instance for indexing, searching, and analysis. By automating its deployment, we guaranteed that all servers would have comprehensive monitoring and logging from their initial boot, which is critical for troubleshooting, security incident response, and operational visibility.

5. **Sapphire IMS:**

   An enterprise-grade IT Service Management (ITSM) solution used for asset management, service desk operations, and IT automation. Automating the installation of the Sapphire agent ensures that every new server is automatically discovered, inventoried, and brought under the management of the IT support framework, streamlining asset tracking and support processes.

Implementation Workflow

Our implementation followed a logical and scalable "control node" architecture. We designated a central Linux VM as our Ansible Host, which acted as the command center for all configuration management tasks. From this single host, we could securely connect to and manage any number of newly provisioned Windows "guest" VMs.

The core of the automation was a library of Ansible **Playbooks** written in YAML. Each playbook was designed to be a self-contained unit of automation for a specific application. A playbook consists of one or more **tasks**, where each task is a single action to be performed.

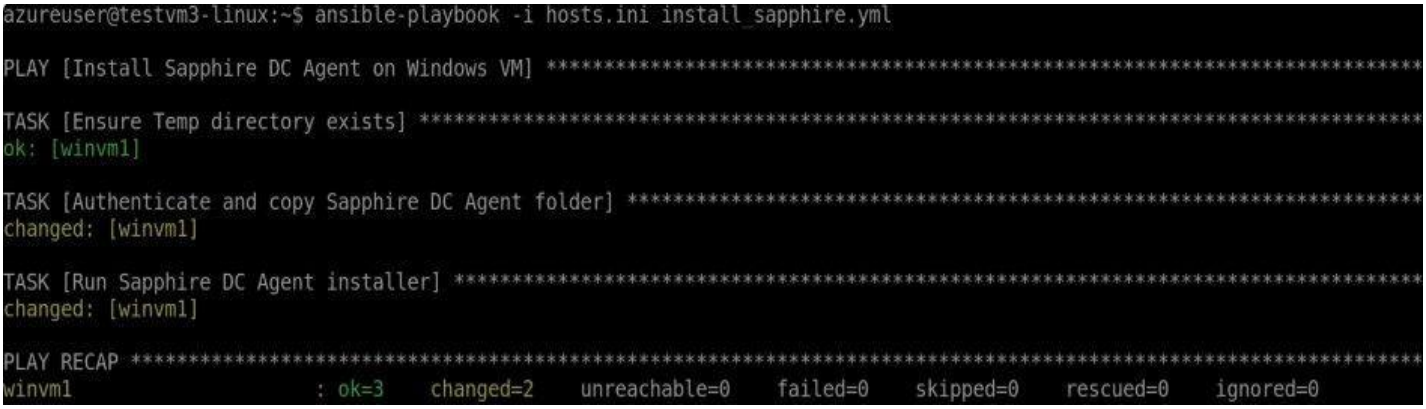
For example, the playbook to install Sapphire contained multiple tasks:

1. A task to download the Sapphire `.msi` installer.
2. A task to run the installer with silent arguments.
3. A task to ensure the Sapphire service was started and enabled on boot.

This modular, task-based approach made the automation easy to read, troubleshoot, and maintain. To configure a new server, an engineer would simply add the server's IP address to the Ansible inventory file on the host VM and run the relevant playbook command **(e.g., `ansible-playbook -i inventory.ini install_splunk.yml`).**

Using this workflow, we successfully automated the deployment and configuration of:

1. **Google Chrome**
2. **CrowdStrike Falcon**
3. **Sapphire IMS**
4. **Splunk Universal Forwarder**

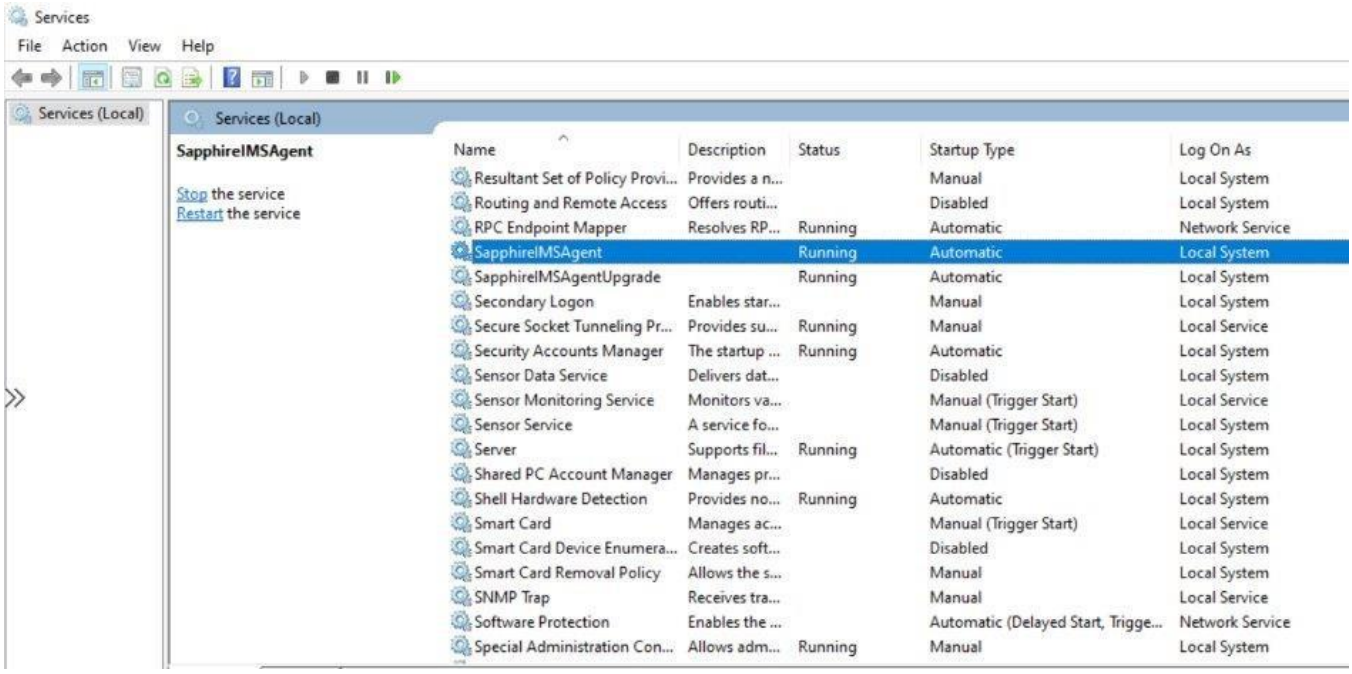


Results and Outcome:

The integration of Ansible for automated configuration management was a critical second step that built upon our infrastructure automation, delivering several key benefits:

1. **End-to-End Automation:**
   We achieved a true end-to-end automation pipeline. A new server could now be provisioned by Terraform and then immediately handed off to Ansible to be fully configured and made application-ready, all without any manual intervention.
2. **Guaranteed Consistency:**
   Ansible's idempotent nature ensures that every server is configured identically every time a playbook is run. This eliminated configuration drift at the application layer, ensuring that all servers in a cluster had the exact same software versions and security settings.
3. **Enhanced Security and Compliance:**
   By codifying the installation and configuration of security agents like CrowdStrike and monitoring tools like Splunk, we ensured that 100% of our servers were compliant with UPL's security and operational standards from the moment they were created.
4. **Rapid Scalability and Recovery:**
   The ability to automatically configure servers at scale is crucial for disaster recovery and scaling events. With our Ansible playbooks, we could now build and configure a fleet of new servers in minutes, a task that would have previously taken days of manual effort.

Example of Sapphire being installed



Working Details

Week no.	Description
5	1. Focused on Ansible implementation. 2. Set up the necessary networking and credentials for Ansible to communicate with guest VMs. 3. Initiated and tested a playbook to install **Google Chrome**.
6	1. Developed Ansible playbooks for enterprise applications. 2. Automated the installation of the **CrowdStrike agent.** 3. Automated the installation of **Sapphire agent.** 4. Verified and tested the configuration of both agents to ensure they were functioning correctly.
7	1. Started exploring Azure Advisor reports to identify potential savings. 2. Developed and ran an Ansible playbook for installing the **Splunk Universal Forwarder.**

PROJECT 3

COST OPTIMIZATION

Objectives of Work

Background

A direct consequence of successful automation is the ease with which new resources can be created. While this accelerates development, it can also lead to an unintended side effect: cloud cost sprawl. Without a corresponding focus on financial governance, the speed of automated provisioning can result in a rapid increase in cloud expenditure from underutilized, forgotten, or oversized resources. Recognizing this, the third phase of my project was dedicated to establishing a proactive framework for cloud cost optimization, shifting the team's approach from a reactive, end-of-month bill analysis to a continuous, data-driven cost management practice.

Prevailing Challenge

As UPL's Azure footprint grew, the IT team faced several key challenges in managing cloud costs effectively:

1. **Lack of Visibility and Accountability:** It was difficult to determine the purpose of many resources or identify their owners. This lack of clear tagging and accountability meant that when a resource was no longer needed, it was often left running, needlessly incurring costs.
2. **Resource Sprawl:** The ease of creating resources for temporary tasks, such as development experiments or testing, often resulted in "orphan resources." These were components like unattached disks, old snapshots, or idle public IP addresses that were forgotten after the primary VM was deleted but continued to generate costs.
3. **Systematic Over-provisioning:** To avoid performance issues, resources were often provisioned with more capacity than they required (e.g., larger VM sizes). This "just-in-case" approach, while safe, was financially inefficient, as a significant portion of the provisioned capacity went unused.
4. **Reactive Cost Management:** The primary method for tracking costs was reviewing the monthly Azure bill. This reactive approach meant that wasteful spending was only identified after the money had already been spent, leaving little room for proactive intervention.

Strategic Actions

Our strategy was to create a continuous and proactive cost optimization loop by leveraging Azure's native analytics and reporting tools. This involved a two-pronged approach to systematically identify and eliminate waste:

1. **Systematic Review of Azure Advisor Recommendations:** We established a regular process to review the cost-saving recommendations automatically generated by Azure Advisor. This tool uses machine learning to analyze usage patterns and identify clear opportunities for optimization, such as right-sizing underutilized VMs or deleting idle resources.
2. **Developing Custom Analytics with Azure Workbooks:** For more nuanced issues that Azure Advisor might miss, we developed custom Azure Monitor Workbooks. I used orphan scripts to hunt for common sources of waste, such as orphan resources (e.g., disks not attached to any VM) or VMs that were running 24/7 but had near-zero CPU usage outside of business hours.

Project Significance

This project was significant because it introduced a culture of financial accountability, or "FinOps," into the infrastructure team's daily operations. By making cost a visible and manageable metric, we ensured that the benefits of agile automation did not come with uncontrolled spending. This initiative provided the tools and processes necessary to maximize the return on UPL's cloud investment, ensuring that every dollar spent on cloud resources delivered tangible business value. It transformed cost management from an afterthought into a core pillar of our cloud operations strategy.

Implementation Workflow

The implementation involved creating a repeatable, weekly process:

1. **Data Collection:** On a set schedule, I would run the reports from Azure Advisor and execute the custom Azure Monitor Workbooks.
2. **Analysis and Reporting:** I analyzed the raw data to identify the most significant and actionable cost-saving opportunities. The findings were compiled into a clear, concise weekly report that detailed the specific resource, the issue (e.g., "Over-provisioned," "Orphan Disk"), and the recommended action (e.g., "Resize from DS3_v2 to DS2_v2," "Delete").

3. **Verification:** The report was presented to the infrastructure team, who would then take the recommended actions. Like for example-

	Month	Subscription	Resource Type	Resource Name	Action/Remarks	Predicted monthly	Predicted Annual Cos
2	July '25	Connectivity (be2769cc-3f21-476c-a96e-42e6b4d18e1d)	Virtual Machine	hubincsdwvm01-fgt-a	Resize Standard_D16s_v3 to Standard_B16ms	6,593.33	52,746.63
3	July '25	Connectivity (be2769cc-3f21-476c-a96e-42e6b4d18e1d)	Virtual Machine	hubincsdwvm01-fgt-b	Resize Standard_D16s_v3 to Standard_B16ms	6,593.33	52,746.63
4	July '25	Connectivity (be2769cc-3f21-476c-a96e-42e6b4d18e1d)	Virtual Machine	hubinsfwvm01	Resize Standard_DS3_v2 to Standard_D2s_v3	10,551.65	84,413.20
5	July '25	dna-cp-dev (8b7cee92-0e07-4870-9ca4-36274b7c1ccc)	Virtual Machine	chatdemo	Resize Standard_D4s_v3 to Standard_D2s_v3	11,133.81	89,070.48
6	July '25	dna-cp-dev (8b7cee92-0e07-4870-9ca4-36274b7c1ccc)	Virtual Machine	eams4wdap01	Resize Standard_D16ads_v5 to Standard_D8ads_v	46,525.82	372,206.53
7	July '25	dna-cp-dev (8b7cee92-0e07-4870-9ca4-36274b7c1ccc)	Virtual Machine	eams4wdap02	Resize Standard_D4as_v5 to Standard_D2as_v5	10,505.88	84,047.00
8	July '25	dna-cp-dev (8b7cee92-0e07-4870-9ca4-36274b7c1ccc)	Virtual Machine	euwe4ldqdmg01	Resize Standard_D4s_v5 to Standard_B4als_v2	3,389.37	27,114.93
9	July '25	dna-cp-dev (8b7cee92-0e07-4870-9ca4-36274b7c1ccc)	Virtual Machine	euwe4wdqdg01	Resize Standard_D8as_v5 to Standard_D2as_v5	31,517.77	252,142.19
10	July '25	dna-cp-dev (8b7cee92-0e07-4870-9ca4-36274b7c1ccc)	Virtual Machine	chatdemo-linux	Shut down the virtual machine	6,039.91	48,319.28
11	July '25	dna-cp-dev (8b7cee92-0e07-4870-9ca4-36274b7c1ccc)	Virtual Machine	aks-edge	Shut down the virtual machine	5,574.57	44,596.55
12	July '25	dna-cp-dev (8b7cee92-0e07-4870-9ca4-36274b7c1ccc)	Virtual Machine	test-edge-2-cp-dev	Shut down the virtual machine	6,039.91	48,319.28
13	July '25	dna-cp-dev (8b7cee92-0e07-4870-9ca4-36274b7c1ccc)	Virtual Machine	test-edge-cp-dev	Shut down the virtual machine	10,933.82	87,470.59
14	July '25	dna-global-core (2d639e07-f6a8-438a-8747-4e56d6f694d0	Virtual Machine	dnatest-edge	Resize Standard_D4s_v3 to Standard_D2s_v3	5,466.91	43,735.29
15	July '25	dna-in-core (a162740e-4a4c-456a-8279-6469e0babb43)	Virtual Machine	test-edge	Shut down the virtual machine	9,567.84	76,542.72
16	July '25	dna-in-mfg-dev (4620089a-8c7f-4423-9157-640693960881	Virtual Machine	aks-edge-mfg	Resize Standard_D2as_v5 to Standard_B2ats_v2	2,620.93	20,967.46
17	July '25	dna-sandbox (14db0b32-db76-4f57-8d04-86115eecaf6d)	Virtual Machine	apci4wdir01	Resize Standard_E4as_v5 to Standard_E2as_v5	8,737.08	69,896.60
18	July '25	dna-sandbox (14db0b32-db76-4f57-8d04-86115eecaf6d)	Virtual Machine	ipnq4tvm02	Shut down the virtual machine	2,401.33	19,210.67
19	July '25	UPL Apps (a122f7bd-3fa7-4118-b809-fe7496405896)	Virtual Machine	ibom4lpfwapp01	Resize Standard_D8s_v3 to Standard_D4s_v3	13,185.91	105,487.29
20	July '25	UPL Apps (a122f7bd-3fa7-4118-b809-fe7496405896)	Virtual Machine	ipnq4wpap34	Resize Standard_D4as_v5 to Standard_B4as_v2	8,976.70	71,813.62

Technologies Used

1. **Azure Advisor:** This served as our first line of defense against common sources of cloud waste. We used it as an automated consultant that provided clear, data-backed recommendations for right-sizing and shutting down underutilized resources, which were often the easiest and quickest cost-saving wins.
2. **Azure Monitor Workbooks:** These were essential for our custom analysis. I used Workbooks to create interactive, visual dashboards that brought our KQL query results to life, making it easy to spot trends and identify specific wasteful resources without having to sift through raw log data.

Results and Outcome

This proactive and data-driven approach to cost management yielded immediate and significant financial benefits:

1. **Identified Significant Savings:** Through this process, we identified potential savings of over **5% of the monthly Azure spend that's about Rs 5M** on compute and storage resources.
2. **Reduced Waste from Orphan Resources:** Our custom custom scripts successfully identified and led to the cleanup of hundreds of orphan resources, reducing monthly costs from this category.
3. **Established Continuous Optimization:** The project successfully established a repeatable, weekly process for cost review and optimization, shifting the team from a reactive to a proactive cost management posture.

Working Details

Week no.	Description
8	1. Used **Azure Workbook Scripts** and **Azure Advisory reports** to extract a list of all orphan resources in the subscription. 2. Documented the orphan resources and the details. Potential **annual saving of about 5M** on UPL's **UPLApps** cloud subscription.
9	1. **Documented all Terraform and Ansible code files** for final submission. 2. Received feedback and completed the **final review** for the internship.

My Learnings

My internship at UPL was a transformative experience, providing deep, hands-on expertise in cloud automation and the principles of modern IT infrastructure management. This was supported by targeted Knowledge Transfer (KT) sessions on core concepts like Infrastructure as Code (IaC), advanced Azure services, and automation best practices with Terraform and Ansible. This foundational knowledge was immediately applicable to my project, allowing me to move from theory to practical implementation swiftly.

I gained a comprehensive understanding of the entire infrastructure lifecycle, from initial requirement gathering to final deployment and optimization. My manager was instrumental in this, initially providing the project scope. However, my role evolved beyond simple execution; I was encouraged to think critically about the implementation. For instance, I actively participated in discussions on how to structure our Terraform modules for better reusability and how to write idempotent Ansible playbooks to ensure consistent configurations.

This proactive approach was fostered by a culture of regular feedback. Reviewing terraform plan outputs and refining automation logic with my team became a crucial part of the workflow, teaching me the importance of clear communication and peer review in a DevOps environment. Ultimately, this experience instilled in me the discipline of writing clean, modular, and well-documented automation code—treating infrastructure not as a set of manual configurations, but as a version-controlled, maintainable product.

The last part of the project Cost-optimization proved one of the most challenging and interesting part of the internship, because of the fact that we had to communicate with other teams and understand the requirements of the environment before making any modification in the existing cloud environment. Running orphan scripts on such large cloud environment was truly rewarding.

Activities Conducted

The internship was packed with interactive sessions, helping me understand the firm and connect with fellow interns.

1. **Induction Orientation:**
   Orientation connected my project to UPL's global strategy. I learned that the company's values of being "Agile" and believing "Nothing's Impossible" depend on a strong IT backbone. This framed my automation project as a key initiative to help UPL operate efficiently and innovate on a global scale

2. **Knowledge Transfer (KT) sessions:**
   These sessions revealed who the scope of my project were: other departments like Networking and Cyber security team. I understood that the infrastructure I was automating wasn't just an IT task, but a critical platform that enables other teams to achieve their goals, directly supporting UPL's digital transformation.

3. **Interaction with the team:**
   This was my core practical training. Working daily with my mentors, I translated high-level goals into functional code. Discussing Terraform scripts and debugging Ansible playbooks was essential for building a solution that met the team's technical and security standards for global operations.

4. **Interaction with fellow interns:**
   Conversations with other interns showed me the direct impact of my work. My automation could provide a data science intern with a server in minutes, not days, and my cost-optimization efforts supported the company's bottom line. This reinforced how a strong IT infrastructure enables every other function in the company.

Conclusion

My summer internship at UPL was an incredibly rewarding experience. I had the opportunity to move beyond theoretical knowledge and apply my skills to solve real-world challenges in a global enterprise environment. The project provided me with deep, hands-on experience in cutting-edge cloud and DevOps technologies like Microsoft Azure, Terraform, and Ansible.

I successfully contributed to the modernization of UPL's IT infrastructure by building a robust automation framework that significantly improved deployment speed, system reliability, and operational efficiency. The cost optimization work further delivered tangible financial benefits to the organization. This internship has solidified my passion for cloud computing and infrastructure automation and has equipped me with practical skills that will be invaluable in my future career.

Furthermore, I had the opportunity to engage with interns from esteemed institutions such as IIT BITS and other colleges too. Leveraging the foundational knowledge and practical skills acquired at VJTI, I found myself adept at collaborating seamlessly with diverse teams. The exposure to a wide range of technological stacks during my academic tenure greatly facilitated my integration into the projects at UPL.

References –

1. **UPL Limited Official Website:** https://www.upl-ltd.com/
2. **Microsoft Azure Documentation:** https://docs.microsoft.com/en-us/azure/
3. **HashiCorp Terraform Documentation:** https://developer.hashicorp.com/terraform/docs
4. **Ansible Documentation:** https://docs.ansible.com/
5. **Azure Cost Management and Billing Documentation:** https://docs.microsoft.com/en-us/azure/cost-management-billing/
6. **Azure Advisor Documentation:** https://docs.microsoft.com/en-us/azure/advisor/